
Gaussian Residual Prediction without an EMA Target Network

Leonardo Tonelli^{1,2} Makoto Yamada²

¹EPFL, Lausanne, Switzerland

²Okinawa Institute of Science and Technology (OIST), Okinawa, Japan

Abstract

We study stochastic predictive embedding learning with one shared encoder and no exponential-moving-average (EMA) target network. Our model, Shared-Var, adds AdaSSL’s low-dimensional Gaussian latent variable (the residual) to a LeWorldModel-style predictor, where the SIGReg regularizer, not an EMA target, prevents collapse. It trains with three terms: latent prediction, posterior–prior KL divergence and SIGReg. We compare it with a deterministic shared model, Shared-Det, and with EMA-target adaptations of BYOL and AdaSSL, in five paired replications per model on two synthetic tasks. A ridge readout maps forecasts to physical coordinates, where the Energy Score evaluates them. On Moving-MNIST, Shared-Var scores lower than Shared-Det in all five replications (mean paired difference -0.923 pixels/frame, 95% interval $[-1.064, -0.782]$), although much of this gap is already present when real future embeddings are decoded. Its difference from the AdaSSL reference is uncertain (the paired interval spans zero). Prior sampling beats the prior mean, and its projector features classify digits less accurately than those of the other three models. On an action-conditioned MPI3D task with held-out attribute combinations, none of the ten variational-model runs beats its own no-change (persistence) forecast on a secondary latent score that bypasses the readout; the deterministic models cannot by construction. A post hoc cost analysis finds that Shared-Var stores 37–42% fewer parameters than the AdaSSL reference, because it keeps no EMA copy. It needs 28% more training FLOPs per update on Moving-MNIST, where the posterior encodes a separate clip, and 14% fewer on MPI3D, where it reuses the target encoding.

1 Introduction

Predictive embedding models learn by predicting the representation of a target, such as another view, a masked region or a future observation, from an observed input [Grill et al., 2020, Assran et al., 2023, Maes et al., 2026]. When the input is compatible with several futures, a deterministic predictor must still commit to a single output. Conditional latent-variable models instead infer a latent from the realized future during training and sample a learned prior at inference [Sohn et al., 2015, Denton and Fergus, 2018]. AdaSSL [Zhang et al., 2026] applies this approach in embedding space: in its variational form, a low-dimensional Gaussian latent variable, which we call the residual, enters the predictor, and a KL term matches its posterior to a conditional prior. Its video recipe uses a BYOL-style exponential-moving-average (EMA) target encoder with a stop-gradient.

LeWorldModel [Maes et al., 2026] trains a deterministic predictor with one encoder shared by source and target. SIGReg (sketched isotropic Gaussian regularization), the distributional regularizer from LeJEPa [Balestriero and LeCun, 2025], replaces the EMA/stop-gradient asymmetry as the mechanism against representational collapse. We combine this shared recipe with AdaSSL’s Gaussian residual. The resulting model, Shared-Var, has one encoder and three loss terms: latent prediction, posterior–prior KL divergence and SIGReg (Eq. (1)). Relative to AdaSSL’s recipe, it removes the

EMA target copy and its momentum schedule, and adds SIGReg with its coefficient and estimator settings. Whether this lowers training compute is an empirical question (Section 5.4).

Two contrasts were prespecified: does the residual improve the shared recipe (Shared-Var versus Shared-Det), and how does Shared-Var compare with an EMA model that uses the same residual (Shared-Var versus the AdaSSL reference)? To answer them we train four models, the BYOL reference, the AdaSSL reference, Shared-Det and Shared-Var, which cross two target recipes (EMA or shared) with two prediction families (deterministic or Gaussian residual). Each model is trained in five paired replications on each of two synthetic tasks, giving 40 runs. Embeddings have no physical units, so a ridge readout fitted on real target features maps each model’s predicted embeddings to velocity or object position, and the Energy Score (lower is better) evaluates the resulting forecast distribution. A persistence control forecasts no change. The primary metric and analysis were frozen before main training.

We centre the presentation on Moving-MNIST, where an uncertain velocity change gives a continuous one-to-many prediction task. There, Shared-Var improves the forecasting pipeline over Shared-Det in all five runs and beats a source-pixel constant-velocity forecast in four. Prior sampling improves on the prior mean in every run, while much of the gap to Shared-Det is already present when real future embeddings are decoded. Its difference from the AdaSSL reference remains uncertain, and its projector retains less digit identity than the AdaSSL reference’s. The second task, built from MPI3D, is action-conditioned: a command either fails or succeeds, and test objects have attribute combinations unseen in training. On this task, no run of any model beats its own persistence forecast on a decoder-independent latent score. Section 5.3 summarizes this result, and Appendix F gives the full analysis.

Our contributions are the shared-encoder Gaussian residual construction and a matched empirical assessment of its forecasts, its representations and, post hoc, its training cost. Shared-Var stores fewer parameters than the AdaSSL reference on both tasks. Its training compute per update is higher on Moving-MNIST, where the posterior observes a separate clip, and lower on MPI3D, where the posterior reuses the target encoding. Together, the results show that the construction trains without an EMA target and improves on its deterministic counterpart on one synthetic video task; on the action-conditioned task, no model we trained, Shared-Var included, beats persistence.

2 Related work

Predictive embeddings and collapse. BYOL [Grill et al., 2020] and I-JEPA [Assran et al., 2023], an image joint-embedding predictive architecture (JEPA), use an EMA target encoder with a stop-gradient to train predictive embeddings. Collapse can also be avoided without an EMA target, for example with a stop-gradient alone in SimSiam [Chen and He, 2021] or with variance and covariance penalties in VICReg [Bardes et al., 2022]. LeJEPA [Balestriero and LeCun, 2025] uses SIGReg, an isotropic-Gaussian regularizer based on the Epps–Pulley test [Epps and Pulley, 1983]. LeWorldModel [Maes et al., 2026] applies SIGReg to an action-conditioned world model with one shared encoder and a deterministic predictor. Our Shared models adapt that recipe with negative cosine prediction, different architectures and fewer SIGReg directions; they are not reproductions of LeWorldModel (Appendices B.4 and B.9).

Latent variables for uncertain futures. Conditional variational models infer a latent from the paired output during training and use a conditional prior at inference [Sohn et al., 2015]. SV2P [Babaeizadeh et al., 2018], SVG [Denton and Fergus, 2018] and PlaNet [Hafner et al., 2019] train stochastic predictors through pixel likelihoods. In embedding space, VJ-VCR [Drozdzov et al., 2024] adds a discrete or sparse latent to a variance–covariance-regularized video JEPA with one shared encoder and no EMA target. Unlike Shared-Var’s residual, its latent is inferred from the observed future and has no learned prior to sample. AdaSSL [Zhang et al., 2026] adds a latent variable to the predictor of a self-supervised embedding objective. Its variational form, AdaSSL-V, uses a learned Gaussian prior and posterior, and its video model uses an EMA target; the AdaSSL reference is our adaptation of that recipe. Shared-Var places the same residual family in a shared, SIGReg-regularized model.

Probabilistic JEPA models. VJEPA [Huang, 2026] predicts a Gaussian distribution over target embeddings with an EMA target and a fixed prior, and is evaluated mainly for robustness to high-

variance distractors, from Noisy-TV systems to image benchmarks and a control task. PhiNet v2 [Yamada et al., 2026] learns video representations with a categorical stochastic latent, a variational objective with a conditional prior, and an EMA encoder. Var-JEPA [Gögl and Yau, 2026] trains a coupled latent-variable model with an observation-space reconstruction likelihood, without EMA, on simulated and tabular data; in its simulation study it also ablates adding SIGReg to its objective and replacing its KL terms with SIGReg. Shared-Var uses a low-dimensional Gaussian residual, a learned conditional prior and SIGReg, with no EMA target and no reconstruction likelihood. We assess its forecasts with the Energy Score [Gneiting and Raftery, 2007] after mapping predicted embeddings to physical coordinates.

3 Models

3.1 Shared-encoder Gaussian residual prediction

A training example contains an observed input x , a target y , and an auxiliary observation u that is available only during training. MPI3D also supplies a command a ; we omit it on Moving-MNIST. An encoder f , comprising a backbone and projector, maps observations to 128-dimensional embeddings. The predictor g takes the source embedding $z_x = f(x)$, the command if present, and a residual r : a low-dimensional random variable intended to carry the part of the outcome that the source does not determine. A diagonal Gaussian prior $p_\theta(r \mid z_x, a)$ sees only the observed information; a diagonal Gaussian posterior $q_\phi(r \mid z_x, a, z_u)$ also sees $z_u = f(u)$. On Moving-MNIST, u is a later clip of the same trajectory; on MPI3D, $u = y$ (Section 3.3). The residual has two coordinates on Moving-MNIST and one on MPI3D.

Shared-Var uses the same encoder for source and target, $z_y = f(y)$, and trains with

$$\begin{aligned} \mathcal{L} = & -\frac{1}{B} \sum_{b=1}^B \cos(g(z_{x,b}, a_b, r_b), z_{y,b}) \\ & + \frac{\beta}{B} \sum_{b=1}^B \text{KL}(q_\phi(r \mid z_{x,b}, a_b, z_{u,b}) \parallel p_\theta(r \mid z_{x,b}, a_b)) \\ & + \frac{\lambda}{2} [\text{SIGReg}(\{z_{x,b}\}_b) + \text{SIGReg}(\{z_{y,b}\}_b)], \end{aligned} \quad (1)$$

where r_b is a reparameterized posterior draw [Kingma and Welling, 2014], $\beta = 0.001$ and $\lambda = 0.1$. We denote the three unweighted losses by $\mathcal{L}_{\text{pred}}$, $\mathcal{L}_{\text{KL}}$ and $\mathcal{L}_{\text{SIG}}$. The latent prediction term aligns predictions with the model’s own target embeddings. The KL term matches the posterior to the learned prior. SIGReg regularizes the real source and target projector outputs toward an isotropic Gaussian [Balestriero and LeCun, 2025, Maes et al., 2026]. It is applied before cosine normalization, and not to predictions or to the separate Moving-MNIST posterior clip. Source, target and posterior-feature gradients all reach the one encoder. Appendix B gives the Gaussian heads, objective reductions and SIGReg estimator.

3.2 Four-model comparison

Our two EMA references are the BYOL reference (deterministic) and the AdaSSL reference (Gaussian residual), local adaptations of BYOL [Grill et al., 2020] and AdaSSL’s video recipe [Zhang et al., 2026] to our prediction tasks. Both use an online encoder f and a stopped target $z_y = \text{sg}[\bar{f}(y)]$, where $\bar{f}$ is an EMA copy of f . The AdaSSL reference adds the Gaussian residual to the BYOL reference; its posterior consumes differentiable online features, while the alignment target stays stopped. The Shared models use $z_y = f(y)$ with gradients through both views and add SIGReg; Shared-Var adds the Gaussian residual to Shared-Det. Table 1 lists the four objectives, and Figure 1 contrasts the two variational models.

Relative to the AdaSSL reference, Shared-Var removes the EMA encoder, its momentum update and its separate batch-normalization (BN) statistics. It adds SIGReg, its coefficient and fixed estimator settings. We use the same β as the AdaSSL reference and the same λ as Shared-Det, without tuning either for Shared-Var. The EMA/Shared contrast changes target gradients, regularization and BN exposure together, and adding the residual also adds heads and training information. The design therefore compares whole recipes rather than isolating individual mechanisms.

Table 1: Training recipes crossed with prediction families: deterministic (Det) or Gaussian residual (Var). All models use $\mathcal{L}_{\text{pred}}$ plus the listed losses, with $\beta = 0.001$ and $\lambda = 0.1$ where applicable. Parentheses count stored encoders. EMA momentum is 0.996 on Moving-MNIST and increases from 0.996 to 1 on MPI3D. Appendix B.1 specifies target gradients and batch-normalization updates.

Model	Target branch (encoders)	Residual r	Additional losses
BYOL reference	EMA, stopped (2)	none	–
AdaSSL reference	EMA, stopped (2)	Gaussian	$\beta\mathcal{L}_{\text{KL}}$
Shared-Det	Shared, gradients (1)	none	$\lambda\mathcal{L}_{\text{SIG}}$
Shared-Var	Shared, gradients (1)	Gaussian	$\beta\mathcal{L}_{\text{KL}} + \lambda\mathcal{L}_{\text{SIG}}$

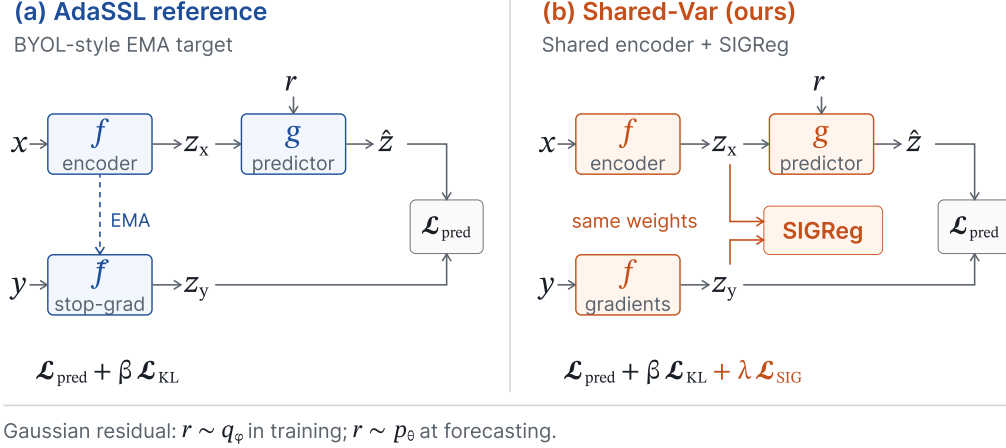


Figure 1: Training paths of the two variational models. (a) The AdaSSL reference uses a BYOL-style EMA target f with stopped target gradients. (b) Shared-Var shares f across source and target, propagates target gradients and adds SIGReg on z_x and z_y . Both use the Gaussian residual r and the posterior–prior KL term. For clarity, the posterior and its input u , the MPI3D command and the evaluation readout are not drawn; Appendix Algorithm 1 gives the complete update.

3.3 Posterior information and forecasting

On Moving-MNIST, the source is frames 1–3, the target is frames 4–6, and the posterior observes frames 7–9 of the same trajectory. The velocity changes once, between frames 3 and 4, and then stays constant (Section 4.1), so the later clip reveals the realized change without being the alignment target. On MPI3D the posterior observes the realized target image: the AdaSSL reference encodes it through an extra online pass, while Shared-Var reuses z_y .

At inference, the variational models encode only the source (and command) and sample the residual from the learned prior; neither the posterior nor u is used. The deterministic models produce one embedding. Each model’s fitted readout maps predicted embeddings to velocity or position (Section 4.3). Because the predictor is nonlinear, the forecast distribution need not be Gaussian even though the residual is. Appendix Algorithms 1 and 2 give one training update and one forecast. Eq. (1) is a latent prediction objective; we derive no image-likelihood bound for it.

4 Experimental design

4.1 Prediction tasks

Moving-MNIST. We follow the single-digit variant of Moving-MNIST [Srivastava et al., 2015] used in AdaSSL’s video experiment [Zhang et al., 2026, App. D.5], with MNIST digits [LeCun et al., 1998]. A 16×16 digit moves on a 64×64 canvas with initial velocity $v \sim \mathcal{U}[0, 3]^2$ px/frame. Between frames 3 and 4, one randomly selected component changes to $v'_k = v_k + \frac{2}{3}v_k\epsilon$, with $\epsilon \sim \mathcal{N}(0, 1)$; the other stays fixed. The source clip is therefore compatible with many future velocities. There is no

bouncing. We read $\frac{2}{3}v_k$ as a standard deviation, a documented local convention for an ambiguity in the reference (Appendix B.9). Training draws digits from 50,000 MNIST training images (identities), and final evaluation uses the 10,000 official test images; the remaining 10,000 training images are used for readout selection and development. Trajectories are generated afresh for every batch. Appendix A gives the rendering, the partitions (Table 5) and an illustration (Figure 3).

MPI3D-S (stochastic command execution). We construct action-conditioned transitions by indexing factor-controlled MPI3D-realistic images [Gondal et al., 2019]; they are not recorded trajectories. The input is an object image and one of four commands (left, right, up or down). A successful command moves the object four grid indices; a failed one leaves the image unchanged. The two outcomes are equally likely, enforced by a balanced training quota. Test objects combine colour, shape and size levels seen during training into combinations never seen in training. Besides its outcome law and split, this task differs from Moving-MNIST in its conditioning and architecture. Appendix A specifies the transition and split construction.

4.2 Training and paired replications

Moving-MNIST uses a 3D convolutional backbone, trained for 75,000 updates at batch 128. MPI3D uses a vision Transformer [Dosovitskiy et al., 2021], trained for 102,400 updates at global batch 64. All models are trained in FP32 with AdamW [Loshchilov and Hutter, 2019], without gradient clipping; architectures and optimization schedules are in Appendix B. Each dataset uses one optimization recipe for all four models, which we did not tune separately for any model. Budgets are matched in updates within each dataset, and every result uses the terminal checkpoint.

Within a replication, all four models share initial weights for their common modules and see the same training examples in the same order. The variational models also share latent noise streams, and the Shared models share SIGReg directions. The five replications vary training randomness on fixed splits and evaluation banks (Appendix B.8). After one development replication per model and dataset, the evaluation configuration, query banks and analysis were frozen before main training. All 40 main runs completed, and none was replaced.

4.3 Evaluation and statistical analysis

Physical forecasts. We score forecasts in physical units. Each checkpoint receives its own ridge readout, fitted and selected on real target-space features and never on forecasts (Appendix C). The readout maps predicted embeddings to the post-change velocity v' (px/frame) on Moving-MNIST and to object position (grid indices) on MPI3D. The primary metric therefore evaluates the encoder, predictor and readout together.

For a forecast distribution P and the true conditional distribution Q , we use the Energy Score in its lower-is-better form [Gneiting and Raftery, 2007]:

$$\text{ES}(P, Q) = \mathbb{E}_{X \sim P, Y \sim Q} \|X - Y\|_2 - \frac{1}{2} \mathbb{E}_{X, X' \sim P} \|X - X'\|_2. \quad (2)$$

The score is strictly proper: it is minimized at $P = Q$, where it equals $\frac{1}{2} \mathbb{E}_{Y, Y' \sim Q} \|Y - Y'\|_2$, so even a perfect forecast of a stochastic outcome scores above zero. On Moving-MNIST, we compare 64 prior draws with 128 independent simulator draws for each of 1,024 queries. On MPI3D, we represent each prior by 16 equal-weight quantiles and enumerate both true outcomes for each of 2,048 queries. The estimators and their distinction from energy distance [Székely and Rizzo, 2013] are in Appendix C.

Controls and probes. The fixed-prior-mean control replaces prior samples with the prior mean in a trained variational model, without retraining. Target-space persistence forecasts no change: it decodes the model’s own target-space embedding of the source. Pixel constant velocity uses source pixels only; the simulator oracle and the coordinate references use privileged state information (Appendix Table 11). Linear probes assess the trained source representations with the encoder held fixed.

MPI3D latent score. For MPI3D we add a secondary, decoder-independent latent score. Forecasts and both outcome embeddings are unit-normalized, and the Energy Score is computed in the model’s own target space and divided by the distance between the two outcome embeddings. Under this gap

normalization, persistence scores exactly 0.5 and the true two-outcome distribution 0.25. Because a failed command leaves the image unchanged, persistence reproduces the failure outcome exactly. By the triangle inequality, no single-point forecast scores below 0.5, so only a forecast distribution with spread can beat persistence; the deterministic models can at best match it. Because target spaces differ between models, each model is compared only with its own persistence control. All MPI3D queries use held-out attribute combinations, so this score cannot separate a failure to forecast from a failure to generalize to new combinations.

Paired comparisons. We analyse the physical Energy Score separately by dataset. The prespecified contrasts are Shared-Var – Shared-Det and Shared-Var – AdaSSL reference. We report all five values, means, sample SDs and individual paired- t 95% intervals with 4 degrees of freedom. These intervals treat the five replication-level scores as the units. They describe training randomness conditional on the fixed benchmark and are not simultaneous across contrasts. We report no query-level uncertainty: the query banks are fixed across models and replications, and the MPI3D queries are clustered (512 sources, each with four commands). The prespecified analysis prescribes no hypothesis tests, and an interval containing zero does not establish equivalence. All other comparisons are secondary and descriptive.

5 Results

5.1 Moving-MNIST forecast accuracy

Shared-Var improves over Shared-Det in all five replications (Table 2, Figure 2), with a mean paired difference of -0.923 px/frame and 95% interval $[-1.064, -0.782]$. Its mean Energy Score is 57% lower. Against the AdaSSL reference, Shared-Var scores lower in four of five runs, but the paired difference -0.075 has interval $[-0.320, 0.170]$. The observed means are close, and this interval establishes neither superiority nor equivalence. Shared-Var’s weakest run, replication 1, scores 0.923 versus 0.641–0.656 for the other four; all five enter the comparison.

Two references need no learned dynamics: the source-pixel constant-velocity forecast (0.797) and each model’s own target-space persistence forecast. Shared-Var beats the pixel forecast in 4 of 5 runs (all but replication 1) and its own persistence in 5 of 5. The AdaSSL reference beats the pixel forecast in 2 and its own persistence in 2 of 5 runs, and neither deterministic model beats the pixel forecast in any run (Appendix Table 17 and Figure 5). These per-run counts are descriptive and do not replace the paired contrast above. The privileged simulator oracle scores 0.610. The BYOL reference scores worst (3.254), even though decoding its real target embeddings scores 0.632.

5.2 Sampling, readout and representation (secondary)

Sampling helps within each variational model. Replacing prior samples with the prior mean worsens the Energy Score in all five runs of each model, by 0.216 on average for Shared-Var and 0.247 for the AdaSSL reference (Appendix Figure 7). At the end of training, the posterior–prior KL on query trajectories averages 4.94 nats for Shared-Var and 5.87 for the AdaSSL reference, so the posterior has not collapsed onto the prior (Appendix Figure 8). The KL is large mainly because the posteriors are much narrower than the prior (mean posterior and prior standard deviations 0.64 and 7.58 for Shared-Var); the sampling gain, not the KL, is the evidence that the residual affects forecasts. Empirical 90% marginal intervals cover 0.81/0.88 of truth draws (x/y) for Shared-Var and 0.68/0.58 for the AdaSSL reference. These finite-sample coverage estimates do not establish calibration (Appendix Table 19 and Figure 11).

Much of the gap to Shared-Det is present when real futures are decoded. Shared-Det’s target embeddings encode the new velocity poorly: its readout reaches $R^2 = 0.146/0.163$ (x/y) on real target clips, against 0.904/0.976 for Shared-Var. Decoding real future embeddings already gives an Energy Score of 1.278 for Shared-Det versus 0.652 for Shared-Var (Appendix Tables 17 and 18). That difference, 0.626 px/frame, is 68% of the gap in mean primary scores. Even Shared-Var’s fixed-prior-mean forecasts (0.918) outperform Shared-Det. The primary comparison therefore measures a change in the whole forecasting pipeline: the learned target representation and its readout as well as forecast spread. The design does not separate these effects.

Table 2: Moving-MNIST physical-forecast Energy Score ($\downarrow$, px/frame): mean $\pm$ sample SD and range over five paired training replications. Bold marks the lowest observed mean; it does not establish superiority. Appendix Tables 14 and 16 give both datasets and all 40 runs.

Model	Energy Score $\downarrow$ (mean $\pm$ SD)	Range
BYOL reference	3.254 $\pm$ 0.245	2.866–3.537
AdaSSL reference	0.777 $\pm$ 0.100	0.665–0.881
Shared-Det	1.625 $\pm$ 0.027	1.596–1.659
Shared-Var	0.702 $\pm$ 0.124	0.641–0.923

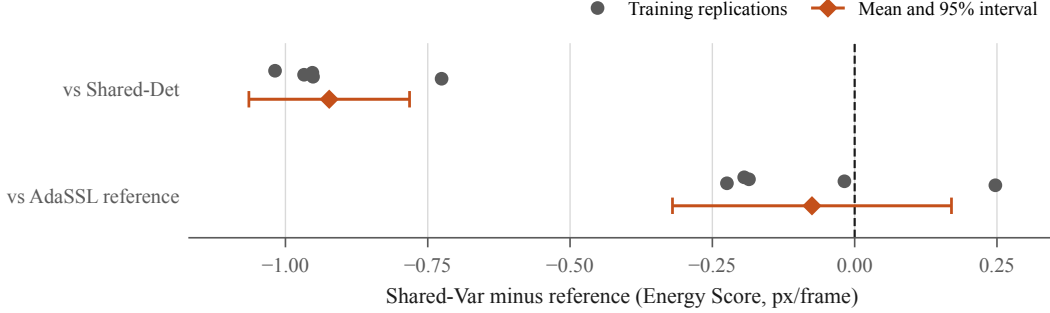


Figure 2: Moving-MNIST paired differences: Shared-Var minus reference. Circles show the five matched training replications; diamonds and bars show the mean and individual paired- t 95% interval (4 degrees of freedom). Negative values favour Shared-Var; zero denotes equal scores. Both comparisons use the same scale.

Motion remains decodable, with less digit identity. Linear digit accuracy on source-clip projector features is 0.517 for Shared-Var, against 0.621 for Shared-Det and 0.852 for the AdaSSL reference; on backbone features the differences are smaller (0.843 against 0.878 and 0.891). Velocity remains linearly decodable from Shared-Var’s projector ($R^2 = 0.920/0.992$, x/y). Relative to Shared-Det, the forecasting gain thus comes with less digit identity at the projector. Appendix E gives all probes (Figure 9), readout fidelity (Figure 10) and speed-stratified results (Figure 12).

5.3 MPI3D: no model beats its own persistence forecast on the latent score (secondary)

MPI3D tests action-conditioned prediction with two discrete outcomes on held-out attribute combinations (Table 3). On the primary physical score, Shared-Var is lower than Shared-Det in only 2 of 5 replications (mean difference -0.87 grid indices, interval $[-3.73, 1.99]$). Its large difference from the AdaSSL reference, -4.144 with interval $[-6.479, -1.809]$, does not reflect forecasting skill: the EMA models’ predicted features leave the readout’s fit support and produce extreme physical scores. Every run scores far above privileged coordinate persistence (2.00). Shared-Var’s readout also transfers less well to the test attributes than the others ($R^2 = 0.737$ on real test targets, against 0.896–0.930), so readout error limits its physical scores.

On the decoder-independent latent score, all 20 runs score worse than their own persistence control, and Shared-Var averages 0.750 against 0.5 for persistence and 0.25 for the true outcome distribution. Only the variational models can beat this control (Section 4.3), and none of their ten runs does (Appendix F, Figure 13). In physical units, Shared-Var scores lower than its decoded persistence embedding in 3 of 5 runs, but readout error dominates both: decoding the real outcome images already scores 4.88, against 1.00 for the exact distribution, so we base the MPI3D conclusion on the latent score. Because every query uses unseen attribute combinations, the failure may reflect poor compositional generalization as well as poor forecasting; the evaluation cannot separate the two.

5.4 Fewer stored parameters, dataset-dependent training cost (post hoc)

Cost was not a prespecified outcome. We counted FLOPs and parameters and ran the timing benchmark after the main runs; the development wall-clock was recorded from the development

Table 3: MPI3D forecasts over five replications per model ($\downarrow$ lower is better): physical score as mean $\pm$ sample SD, latent score as mean (min–max). Physical scores are in Euclidean grid indices; “own persistence” decodes the model’s target-space embedding of the source. The latent score is computed in each model’s own target space, so it compares each model only with its own persistence (0.5 by construction) and with the true two-outcome distribution (0.25); it does not rank the models. All queries use held-out attribute combinations. Appendix Table 20 gives all controls.

Model	Physical ES $\downarrow$ (grid indices)		Latent ES $\downarrow$ (gap-normalized)
	Sampled prior	Own persistence	Sampled prior
BYOL reference	$4.04 \times 10^6 \pm 4.03 \times 10^6$	3.80	2.396 (2.015–2.861)
AdaSSL reference	$4,150 \pm 1,881$	3.91	1.282 (1.086–1.512)
Shared-Det	7.22 ± 2.06	4.29	0.893 (0.777–1.012)
Shared-Var	6.35 ± 2.23	5.92	0.750 (0.600–0.913)
Coordinate persistence (privileged)		2.00	–
Two-outcome oracle (privileged)		1.00	0.250

Table 4: Shared-Var relative to the AdaSSL reference (post hoc, secondary). Negative percentages indicate reductions. FLOPs are analytic matrix-multiplication and convolution counts per training update; stored parameters include the EMA copy, and trainable parameter counts are equal. Equal update budgets give a recipe comparison rather than a cost-matched accuracy comparison. Appendix D gives the absolute values, counter scope and timing measurements.

Dataset	Stored parameters	Training FLOPs / update
Moving-MNIST	–37%	+28%
MPI3D	–42%	–14%

runs before main training. Removing the EMA copy reduces stored parameters on both datasets (Table 4), but it does not consistently reduce training compute. The difference follows the posterior input. On Moving-MNIST, Shared-Var backpropagates through three encoder passes (source, target and the separate posterior clip), whereas the AdaSSL reference backpropagates through two and runs its EMA target pass without gradients. Shared-Var therefore uses 28% more training FLOPs per update. On MPI3D, Shared-Var reuses the target encoding as the posterior input and needs only the two passes of Shared-Det. It uses 14% fewer FLOPs than the AdaSSL reference and 0.1% more than Shared-Det. The timing measurements agree in sign (Appendix D, Figure 4). Per-forecast inference compute is the same for the two variational models. The MPI3D saving accompanies forecasts that fail the persistence control.

6 Discussion and limitations

A shared encoder with SIGReg can support a Gaussian residual predictor on Moving-MNIST. Relative to Shared-Det, the gain combines forecast spread with a more decodable target representation. Relative to the AdaSSL reference, the observed mean scores are close, but five replications leave the difference uncertain. On MPI3D, none of the ten variational-model runs beats its own persistence forecast on the latent score, and the deterministic models cannot by construction.

The four-model design changes several components together. EMA versus Shared alters target gradients, regularization and BN exposure; deterministic versus Gaussian residual prediction adds capacity and posterior information during training. We cannot attribute the results to randomness, SIGReg or the removal of EMA alone. The primary metric includes a fitted linear readout, and its distortion contributes to Shared-Det’s Moving-MNIST error and to the MPI3D scores. Other readouts or prediction normalization could change the results. The two tasks also use different architectures and posterior inputs, so differences between them cannot be attributed to the data alone.

Five replications describe training randomness on fixed splits and evaluation banks, and the analysis includes no equivalence test. Both tasks are synthetic, with single-step, low-dimensional targets. Each dataset uses one optimization recipe for all four models, and we did not retune it for the Shared models. Equal update budgets need not give equal convergence: we could not inspect training curves,

and on MPI3D the learning rate is still about 11% of its peak at the final update. We did not test sensitivity to β , λ or the SIGReg settings, or establish Monte Carlo and quadrature convergence of the scores.

On Moving-MNIST, the AdaSSL reference follows the published AdaSSL video recipe with documented local conventions, because the official repository, at the commit we pinned, contains no video pipeline (Appendix B.9). On MPI3D, both EMA references are our own image-and-command adaptations, so their failure there should not be attributed to the published methods.

The MPI3D split was inspected during earlier development experiments that are not reported here, and earlier use of the MNIST test set cannot be ruled out, although the development protocol recorded no test access. Checkpoints, fitted readouts, per-query outputs and training curves were not available for this analysis, so it uses exported run-level results only and shows no forecast examples or training curves. We checked the method descriptions against the experiment code and configuration records, but could not verify byte for byte that this code is the snapshot that was executed.

7 Conclusion

Shared-Var combines Gaussian residual prediction with one shared encoder and a three-term objective: latent prediction, KL divergence and SIGReg. On Moving-MNIST, adding the residual to the shared recipe lowers the physical Energy Score in all five replications, and much of this gap is already present when real future embeddings are decoded, while the difference from the AdaSSL reference remains uncertain. On MPI3D, the residual gives no clear improvement over Shared-Det, and none of the ten variational-model runs beats its own persistence forecast on the gap-normalized latent score; the deterministic models cannot by construction. Shared-Var stores fewer parameters than the AdaSSL reference on both tasks. Its training compute per update was higher on Moving-MNIST, where the posterior needs a separate observation, and lower on MPI3D, where it reuses the target encoding. Natural next experiments would test prediction-scale control on MPI3D, score MPI3D forecasts on training attribute combinations to separate forecasting from compositional generalization, measure how often weak Moving-MNIST runs occur, and test whether digit identity can be preserved alongside the forecasting gains.

Reproducibility statement

Appendices A–C specify the data, architectures and evaluation. All tables and figures are generated by a script that recomputes the primary analysis from the exported run results, checks it against the exported values, and records the plotted values and input hashes. The code and exported results are available at <https://github.com/leonardotonelli/gaussian-residual-prediction>.

Acknowledgments and Disclosure of Funding

This work was carried out during Leonardo Tonelli’s summer internship at OIST, with no funding or support external to OIST.

References

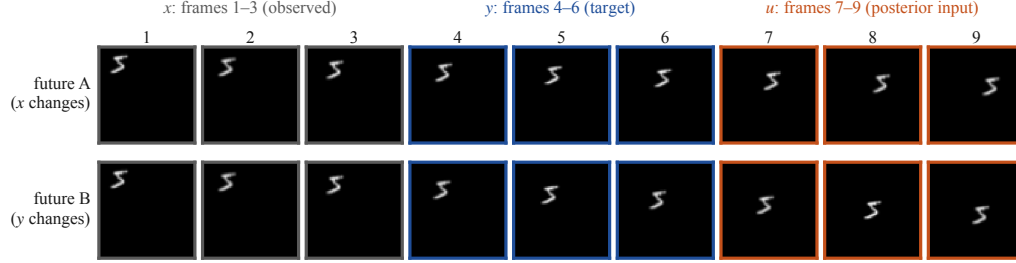
- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023. doi: 10.1109/CVPR52729.2023.01499.
- Mohammad Babaeizadeh, Chelsea Finn, Dumitru Erhan, Roy H. Campbell, and Sergey Levine. Stochastic variational video prediction. In *International Conference on Learning Representations (ICLR)*, 2018.
- Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics, 2025. arXiv:2511.08544v3.

- Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2105.04906.
- Xinlei Chen and Kaiming He. Exploring simple Siamese representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15745–15753, 2021. doi: 10.1109/CVPR46437.2021.01549.
- Emily Denton and Rob Fergus. Stochastic video generation with a learned prior. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1174–1183, 2018.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- Katrina Drozdov, Ravid Shwartz-Ziv, and Yann LeCun. Video representation learning with joint-embedding predictive architectures, 2024. arXiv:2412.10925v1.
- T. W. Epps and Lawrence B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983. doi: 10.1093/biomet/70.3.723.
- Tilman Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.
- Moritz Gögl and Christopher Yau. Var-JEPA: A variational formulation of the joint-embedding predictive architecture – bridging predictive and generative self-supervised learning. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 35490–35513, 2026. arXiv:2603.20111v2.
- Muhammad Waleed Gondal, Manuel Wüthrich, Djordje Miladinovic, Francesco Locatello, Martin Breidt, Valentin Volchkov, Joel Akpo, Olivier Bachem, Bernhard Schölkopf, and Stefan Bauer. On the transfer of inductive bias from simulation to the real world: a new disentanglement dataset. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent – a new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 21271–21284, 2020.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2555–2565, 2019.
- Yongchao Huang. VJEPA: Variational joint embedding predictive architectures as probabilistic world models. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 47703–47731, 2026. arXiv:2601.14354.
- Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014. arXiv:1312.6114.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels, 2026. arXiv:2603.19312v3.

- Kihyuk Sohn, Honglak Lee, and Xinchun Yan. Learning structured output representation using deep conditional generative models. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- Nitish Srivastava, Elman Mansimov, and Ruslan Salakhutdinov. Unsupervised learning of video representations using LSTMs. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 843–852, 2015.
- Gábor J. Székely and Maria L. Rizzo. Energy statistics: A class of statistics based on distances. *Journal of Statistical Planning and Inference*, 143(8):1249–1272, 2013. doi: 10.1016/j.jspi.2013.03.018.
- Makoto Yamada, Kian Ming A. Chai, Ayoub Rhim, Satoki Ishikawa, Mohammad Sabokrou, and Yao-Hung Hubert Tsai. Brain-inspired stochastic joint embedding representation learning, 2026. arXiv:2505.11129v2.
- Yipeng Zhang, Hafez Ghaemi, Jungyoon Lee, Shahab Bakhtiari, Eilif B. Muller, and Laurent Charlin. Self-supervised learning from structural invariance. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2602.02381v2.

A Dataset laws and partitions

(a) Moving-MNIST: one observed clip, two sampled futures



(b) MPI3D-S: source image and command right; the command fails or succeeds with probability 1/2



Figure 3: What the models observe and predict. (a) Moving-MNIST, rendered by the generator used for all runs (training identity `mnist-train:00000`, digit 5). The source velocity is (2.61, 1.82) px/frame, and the two rows show two sampled futures with new velocities (3.76, 1.82) and (2.61, 3.32). The example is illustrative; its seed rule is given below. (b) MPI3D-S: the stochastic-execution row of our rendering of the official archive, cropped without restyling. Source factors are colour 4, shape 3, size 1, camera 1 and position (20, 20), with the command right. The residue $(4 + 3 + 3 \cdot 1) \bmod 6 = 4$ makes this a *validation-split* object.

A.1 Moving-MNIST

Law and rendering. Coordinates are (x, y) in pixels, with the origin at the top-left pixel centre and y increasing downward; velocities are in px/frame. For a source state with centre c_0 and velocity v ,

$$c_0 \sim \mathcal{U}[8, 16]^2, \quad v \sim \mathcal{U}[0, 3]^2, \quad k \sim \text{Bernoulli}(\tfrac{1}{2}), \quad \epsilon \sim \mathcal{N}(0, 1),$$

$$v'_k = v_k + \tfrac{2}{3}v_k \epsilon, \quad v'_{1-k} = v_{1-k},$$

$$c_t = c_0 + (t-1)v \quad (t = 1, 2, 3), \quad c_t = c_0 + 2v + (t-3)v' \quad (t = 4, \dots, 9).$$

Here $k = 1$ means that the vertical component changes. A 28×28 digit is scaled to $[0, 1]$, resized to 16×16 by antialiased bilinear interpolation (`align_corners=False`), and placed by bilinear grid sampling with zero padding. A digit centred at c has its leftmost pixel centre at $c - 7.5$. Frames outside the canvas are cropped. There is no bounce, and velocities are never clipped or resampled. Clips have shape (1, 3, 64, 64): x is frames 1–3, y frames 4–6, and u frames 7–9. The Gaussian scale is read as a standard deviation. Under a variance reading the conditional variance on the changed axis would be $\frac{2}{3}v_k$ instead of $(\frac{2}{3}v_k)^2$, which would need a different generator (Appendix B.9).

Identity manifest. The identity manifest has content hash `fd4a5cab...1784`, equal to the value recorded in the evaluation protocol fixed before main training. It permutes the official training indices with numpy PCG64 seed 0. The first 50,000 indices (stored sorted) become training identities, the remaining 10,000 become development identities, and all 10,000 official test images become test identities. Identifiers are namespaced (`mnist-train:NNNNN`, `mnist-test:NNNNN`), so indices cannot collide across the two files. Before any data access, the loader verifies the raw MNIST files against the checksums recorded in the manifest. Table 6 maps these identity populations to the evaluation banks.

Table 5: Dataset construction and evaluation populations derived from the generator, the split code and the evaluation protocol.

Moving-MNIST	
Image source	MNIST, 28×28 digits
Identity partition	50,000 train / 10,000 development (5,000 selection, 5,000 report) / 10,000 official test
Training examples	fresh online clips; 75,000 updates × 128 = 9.6M presentations
Observed x / target y	frames 1–3 / frames 4–6
Posterior input u (training)	frames 7–9 of the same trajectory
Readout and probe fit	8,192 clips, train identities
Readout and probe selection	2,048 clips, development selection identities
Final report / forecast queries	2,048 / 1,024 clips, official test identities
Draws per query	64 forecast, 128 physical truth
MPI3D-S (balanced stochastic execution)	
Image source	MPI3D-realistic, 64×64, background fixed
Attribute split	48 / 12 / 12 colour–shape–size combinations
Training population	589,824 source/command pairs (48 × 3 cameras × 1,024 positions × 4)
Training examples	100 epochs × 65,536 = 6.55M presentations, 102,400 updates
Observed x, a / target y	source image and unit command / realized next image
Posterior input u (training)	the realized target y itself
Readout fit / select / hold-out	8,192 / 2,048 / 2,048 train-attribute images (of 230,400)
Probe fit / select / report	8,192 train / 2,048 validation / 2,048 test images
Forecast queries	2,048 = 512 test sources × 4 commands (of 36,864)
Truth and forecast support	exact two outcomes; 16 equal-weight prior quantiles

Table 6: Moving-MNIST bank mapping. All banks are generated online from addressed random streams, with identities drawn with replacement from the listed population. Every run’s readout and probes are fitted and selected on the fit and selection banks. The development report and query banks served only the development replication; the final banks produced every reported main-run score.

Bank	Identity population	Used for	Clips
Fit	50,000 training identities	readout and probe fitting	8,192
Selection	development identities, first half (5,000)	readout penalty/transform; probe early stopping	2,048
Development report / queries	development identities, second half (5,000)	development replication only	2,048 / 1,024
Final report	10,000 official test identities	probe and readout reporting	2,048
Final queries	10,000 official test identities	primary physical score	1,024

Visibility. Truth futures are rendered for frames 4–6 of the first 64 of the 128 truth draws of each final query. None was blank (zero pixel mass). A fraction 1.73×10^{-4} of them were geometrically cropped, meaning that the nominal 16×16 box left the canvas. Cropping therefore affects very few rendered targets in the final bank.

Illustration seed. The example in Figure 3a uses training identity `mnist-train:00000`. Its seed is the first $s = 0, 1, \dots$ such that `SeedSequence([20260930, s])` gives both initial velocity components ≥ 1.2 px/frame, the first x -changed and first y -changed of 16 future draws each change the velocity by ≥ 1 px/frame, and all nine digit centres of both futures lie in $[9, 55]$. That is $s = 24$. The rule was chosen for visibility, and the example is not an evaluation query.

A.2 MPI3D-S

Factors and state. MPI3D-realistic has seven factors: object colour (6), shape (6), size (2), camera height (3), background colour (3), and horizontal and vertical position (40 each). That makes $6 \cdot 6 \cdot 2 \cdot 3 \cdot 3 \cdot 40 \cdot 40 = 1,036,800$ images of 64×64 pixels. The SHA256 of the MPI3D archive used here is recorded in the evaluation protocol (`f6124f35...`). We fix the background at index 0. Camera height varies across examples but never within a transition. Commands are left (−1, 0), right

(1, 0), up (0, −1) and down (0, 1) on the (horizontal, vertical) position indices. Success adds $4a$ to the position, and failure leaves the image unchanged.

Training sampler. Sources are restricted to the common legal region $[4, 35]^2$ (1,024 positions), so every command is legal and targets lie in $[0, 39]^2$. Each epoch takes 65,536 distinct source/command pairs, without replacement, from the 589,824 training pairs (48 attribute combinations $\times$ 3 camera heights $\times$ 1,024 positions $\times$ 4 commands). It assigns exactly 32,768 of them to failure and 32,768 to success, in shuffled order. This is a balanced quota, not a sequence of independent Bernoulli draws. Over 100 epochs the models see 6.55M presentations in 102,400 updates.

Split integrity. We assign attribute combinations using $(\text{colour} + \text{shape} + 3 \text{ size}) \bmod 6$: residues 0–3 train, 4 validation and 5 test. Table 7 lists the split counts. The three attribute sets are disjoint by construction. Position manifests are shared across splits: 64 of the 256 test positions also occur in the probe-train manifest. Images are therefore separated by attribute combination, not by position. Training transitions use all 1,024 legal source positions with training attributes. Readout images use training attributes at all 1,600 positions.

Table 7: MPI3D split counts, computed from the split code and position manifests. The test position manifest’s SHA256 (75fc0370...) equals the value recorded for the final query bank.

	Train	Validation	Test
Attribute combinations (residues)	48 (0–3)	12 (4)	12 (5)
Combinations per colour / shape / size level	8 / 8 / 24	2 / 2 / 6	2 / 2 / 6
Source positions	1,024 (all legal)	128 (manifest)	256 (manifest)
Source/command pairs	589,824	18,432	36,864
Clean probe images	36,864	4,608	9,216
Role in final evaluation	readout and probe fit	probe selection	queries, probe report

Query construction. The final query bank permutes the 9,216 test sources with a fixed evaluation stream, keeps the first 512, and includes all four commands for each. The two outcome images of every query are enumerated in failure-then-success order. Wrong-source controls shift sources cyclically within each evaluation batch, choosing a shift under which no query keeps its own source factors.

Sources and licences. MPI3D is distributed under a Creative Commons Attribution 4.0 licence (official repository `rr-learning/disentanglement_dataset`). MNIST was used through the official distribution files, verified by checksum; we found no licence statement on the official MNIST page.

B Architecture and optimization

B.1 Model details

The EMA models update their target parameters as $\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau)\theta$ once after every optimizer step. Only parameters are averaged; the EMA copy keeps its own BN running statistics, which its training-mode target passes update. In the Shared models every forward pass (source, target and, for Shared-Var on Moving-MNIST, the posterior clip u) updates the one set of BN statistics. The alignment loss is the same negative batch-mean cosine in every model, and it is not scaled by $1 - \lambda$ as in LeJEPa.

The posterior of the AdaSSL reference consumes differentiable online features: $f(x)$ and $f(u)$ on Moving-MNIST, and $f(x)$, a and an extra online encoding $f(y)$ on MPI3D. Only the EMA alignment target is stopped. Shared-Var uses the same heads, with $z_u = f(u)$ on Moving-MNIST and $z_u = z_y$ on MPI3D. The deterministic Moving-MNIST predictor takes z_x alone, while the variational one takes $[z_x, r]$. The MPI3D predictor always takes $[z_x, a, r]$, with $r = 0$ for the deterministic models. The Gaussian heads and, on Moving-MNIST, the wider predictor input add 407,560 trainable parameters on Moving-MNIST and 405,508 on MPI3D.

Table 8: Training recipes. EMA momentum applies to the EMA models, SIGReg to the Shared models and the KL term to the variational models.

	Moving-MNIST	MPI3D
Encoder; projector	3D CNN, 5 layers, 768-d pooled; 768→1024→1024→128	ViT (patch 8, width 256, depth 6, 8 heads), mean-pooled; 256→1024→1024→128
Predictor input; residual	z_x or $[z_x, r]$; $d_r=2$, softplus std	$[z_x, a, r]$ ($r=0$ for Det models); $d_r=1$, clamped log-std
Stored parameters (M), BYOL reference / AdaSSL reference / Shared-Det / Shared-Var	6.03 / 6.44 / 3.67 / 4.08	10.70 / 11.11 / 6.01 / 6.42
Batch; updates	128; 75,000	64 (4 GPUs); 102,400
AdamW learning rate	0.0001, cosine to 0	0.001, 10-epoch warm-up, cosine over 1.25× budget
Weight decay	0.0001, all parameters	0.04→0.4; no decay on norms and most biases (App. B.7)
EMA momentum (EMA models)	0.996, constant	0.996→1, cosine
β (Var models); λ (Shared models)	0.001; 0.1 (128 directions, 17 knots, [0.2, 4])	same

B.2 Layers

Moving-MNIST. The encoder backbone has five Conv3d layers without bias, each followed by BN and ReLU. Channels are $1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 128 \rightarrow 256$. Kernels are (1, 3, 3), (1, 3, 3), (3, 1, 1), (3, 1, 1), (1, 3, 3) with padding $k/2$. Spatial strides are 2, 2, 1, 1, 2 and the temporal stride is always 1. The output has shape (256, 3, 8, 8); averaging over height and width and flattening channels and time gives 768 features. The projector is Linear(768 → 1024)–BN–ReLU–Linear(1024 → 1024)–BN–ReLU–Linear(1024 → 128)–BN, with bias-free linears. The predictor is Linear($d_{\text{in}} \rightarrow 1024$)–BN–ReLU–Linear(1024 → 1024)–BN–ReLU–Linear(1024 → 128), with $d_{\text{in}} = 128$ (deterministic models) or 130 (variational models). Each Gaussian head is Linear($d \rightarrow 1024$)–BN–ReLU–Linear(1024 → 4), initialized Xavier-uniform with ReLU gain; the first Linear has no bias and the output bias starts at zero. It outputs two means and two scale pre-activations.

MPI3D. A patch embedding (Conv2d 3 → 256, kernel and stride 8) gives 64 tokens with learned position embeddings. These pass through six pre-norm Transformer blocks (width 256, 8 heads, MLP width 512, GELU, no dropout) and a final LayerNorm, and are mean-pooled into 256 features. The encoder also contains a CLS token and a CLS output projection that are never used and are excluded from training; they are the 16,704 non-trainable parameters of Shared models. The projector is 256 → 1024 → 1024 → 128 with hidden BN and ReLU and a final BN. The predictor is 131 → 1024 → 1024 → 128 with hidden BN and ReLU and a biased output. The prior (130 → 1024 → 2) and posterior (258 → 1024 → 2) output a mean and a log standard deviation.

Capacity. Stored parameter counts are given in Table 8 and Table 13. Each EMA copy stores 2,359,008 parameters (Moving-MNIST) or 4,692,032 (MPI3D).

Initialization. Within a replication, every model receives identical initial weights for the modules it shares with the others, set by logical module keys. Moving-MNIST’s stochastic predictor has 130 input columns. Its first 128 columns reuse the deterministic predictor’s fan-in-128 initialization, and its two residual columns keep their own fan-in-130 stream. This raises the uniform bound of the common columns by about 0.78% relative to a fan-in-130 default. Initial EMA targets are exact copies of the online encoder, including BN buffers.

B.3 Gaussian residual heads and KL

On Moving-MNIST the prior sees z_x (128 inputs) and the posterior $[z_x, z_u]$ (256 inputs). Each outputs two means and two scale pre-activations, and the standard deviation is $\text{softplus}(\cdot) + 10^{-6}$.

On MPI3D the prior sees $[z_x, a]$ (130 inputs) and the posterior $[z_x, a, z_u]$ (258 inputs). Each outputs a mean and a log standard deviation clamped to $[-5, 2]$. One reparameterized posterior draw is used per training example. The KL term is the closed form

$$\mathcal{L}_{\text{KL}} = \frac{1}{B} \sum_b \sum_{j=1}^{d_r} \left[\log \frac{\sigma_{p,bj}}{\sigma_{q,bj}} + \frac{\sigma_{q,bj}^2 + (\mu_{q,bj} - \mu_{p,bj})^2}{2\sigma_{p,bj}^2} - \frac{1}{2} \right], \quad (3)$$

summed over residual coordinates and averaged over examples. β is constant from the first update, with no warm-up. The prior learns only through this term. The posterior learns through both the KL and the prediction loss, and both of its feature inputs retain gradients; the KL gradient also reaches the encoder through the prior’s input z_x .

B.4 SIGReg estimator

The statistic in Eq. (1) is

$$\text{SIGReg}(\{z_n\}) = \frac{1}{M} \sum_{m=1}^M N \int_{0.2 \leq |t| \leq 4} e^{-t^2/2} |\hat{\varphi}_m(t) - e^{-t^2/2}|^2 dt, \quad (4)$$

where $\hat{\varphi}_m(t) = \frac{1}{N} \sum_n e^{it\langle d_m, z_n \rangle}$ is the empirical characteristic function along direction d_m . It is applied to the real source and target projector outputs, after the output BN and before the cosine normalization, and the two values are averaged. It is not applied to predictions or to the posterior-clip embedding z_u . Numerically, for embeddings $Z \in \mathbb{R}^{N \times 128}$, SIGReg draws $M = 128$ fresh Gaussian vectors at every update, normalizes them to unit directions d_m , and projects $h_{nm} = \langle z_n, d_m \rangle$. At knots $t_k = 0.2 + (k-1)\Delta$ with $\Delta = 3.8/16$, $k = 1, \dots, 17$, it computes

$$e_m(t_k) = \left(\frac{1}{N} \sum_n \cos(t_k h_{nm}) - e^{-t_k^2/2} \right)^2 + \left(\frac{1}{N} \sum_n \sin(t_k h_{nm}) \right)^2.$$

It returns $\frac{1}{M} \sum_m N \sum_k \omega_k e_m(t_k)$, where the trapezoid weights are $\omega_k = \Delta e^{-t_k^2/2}$ at the end knots and $2\Delta e^{-t_k^2/2}$ in the interior. These weights integrate over both signs of t , since $e_m(-t) = e_m(t)$. Embeddings are gathered across data-parallel ranks before the statistic is computed, so N is the global batch. Directions come from dedicated, checkpointed CPU generators, one for the source view and one for the target view. The configuration fixes $\lambda = 0.1$, 128 directions, 17 knots and the interval $[0.2, 4]$. LeWorldModel uses $\lambda = 0.1$ with 1024 directions and trapezoidal quadrature on $[0.2, 4]$ [Maes et al., 2026]. LeJEPa’s pseudocode integrates over $[-5, 5]$ [Balestriero and LeCun, 2025].

B.5 Training and inference procedures

Algorithm 1 One training update. The MPI3D arguments (a) are omitted on Moving-MNIST.

```

1:  $z_x \leftarrow f(x)$  ▷ online / shared encoder, with gradients
2: if model  $\in \{\text{BYOL reference, AdaSSL reference}\}$  then
3:    $z_y \leftarrow \text{sg}[\hat{f}(y)]$  ▷ EMA copy in training mode: own BN batch statistics
4: else
5:    $z_y \leftarrow f(y)$  ▷ shared encoder, with gradients
6: end if
7: if model  $\in \{\text{AdaSSL reference, Shared-Var}\}$  then
8:    $z_u \leftarrow f(u)$  on Moving-MNIST; on MPI3D  $z_u \leftarrow f(y)$  for the AdaSSL reference and  $z_u \leftarrow z_y$  for Shared-Var
9:    $(\mu_p, \sigma_p) \leftarrow p_\theta(z_x, a)$ ;  $(\mu_q, \sigma_q) \leftarrow q_\phi(z_x, a, z_u)$ ;  $r \leftarrow \mu_q + \sigma_q \odot \epsilon$ 
10:   $\mathcal{L} \leftarrow \mathcal{L}_{\text{pred}}(g(z_x, a, r), z_y) + \beta \mathcal{L}_{\text{KL}}$ 
11: else
12:   $\mathcal{L} \leftarrow \mathcal{L}_{\text{pred}}(g(z_x, a, 0), z_y)$  ▷ deterministic models on Moving-MNIST:  $g(z_x)$ 
13: end if
14: if model  $\in \{\text{Shared-Det, Shared-Var}\}$  then
15:   $\mathcal{L} \leftarrow \mathcal{L} + \lambda \cdot \frac{1}{2} [\text{SIGReg}(z_x) + \text{SIGReg}(z_y)]$ 
16: end if
17: AdamW step on  $\nabla \mathcal{L}$ ; for EMA models,  $\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau)\theta$  (parameters only); scheduler step

```

Algorithm 2 Forecast for one query (all modules in evaluation mode, parameters fixed).

```
1:  $z_x \leftarrow f(x)$ 
2: if variational model then
3:   Moving-MNIST:  $r^{(m)} \sim p_\theta(\cdot | z_x)$ ,  $m = 1..64$ ; MPI3D:  $r_k = \mu_p + \sigma_p \Phi^{-1}((k - \frac{1}{2})/16)$ ,  $k = 1..16$ ,
   weights  $\frac{1}{16}$ 
4: else
5:   one forecast with  $r = 0$  (or no residual input)
6: end if
7:  $\hat{z}^{(m)} \leftarrow g(z_x, a, r^{(m)})$ ;  $\hat{s}^{(m)} \leftarrow WS(T(\hat{z}^{(m)})) + b$  ▷ T: raw/unit transform; S: standardization
```

Table 9: Encoder passes per update, gradient paths and BN exposure. “BN updates” counts the training-mode passes that update each encoder’s running statistics per optimizer step (online or shared encoder; EMA copy).

Model	Moving-MNIST passes	MPI3D passes	Encoder gradients from	BN updates
BYOL reference	$f(x); \bar{f}(y)$	$f(x); \bar{f}(y)$	source	1; EMA 1
AdaSSL reference	$f(x), f(u); \bar{f}(y)$	$f(x), f(y); \bar{f}(y)$	source, posterior input	2; EMA 1
Shared-Det	$f(x), f(y)$	$f(x), f(y)$	source, target	2
Shared-Var	$f(x), f(y), f(u)$	$f(x), f(y)$ (reused as z_u)	source, target and, on Moving-MNIST, posterior clip	3 on Moving- MNIST, 2 on MPI3D

B.6 Gradient and batch-normalization policy

The Moving-MNIST trainer verifies these counts at every checkpoint and refuses to save if an extra training-mode pass has occurred. The MPI3D trainer verifies them whenever a run resumes, and the post-training verification checks them at the terminal checkpoint of every MPI3D run. On MPI3D, BN layers are converted to synchronized BN over four ranks. No model shares a joint BN pass over concatenated views, and no dummy passes were added to equalize exposure.

B.7 Optimization

On Moving-MNIST, AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) uses learning rate and weight decay 10^{-4} on all trainable parameters, including biases and BN affine terms. The learning rate follows cosine annealing to zero over 75,000 updates, with no warm-up, clipping, accumulation or mixed precision. Each step runs the optimizer, then the parameter-only EMA update (EMA models), then the scheduler. Neither dataset uses data augmentation; Moving-MNIST trajectories are generated afresh for every batch. On MPI3D, AdamW with the same betas and default ϵ uses a peak learning rate of 10^{-3} . The learning rate rises linearly over 10 epochs (10,240 updates) and then follows a cosine toward zero over a horizon of $1.25 \times 102,400 = 128,000$ updates, so at the final update it is still about 11% of its peak. Weight decay follows a cosine from 0.04 to 0.4 over the actual 102,400 updates, with no decay on normalization parameters or on parameters named bias; the attention input-projection biases (4,608 parameters) are decayed. EMA momentum follows a cosine from 0.996 to 1 over the same range. The global batch is 64 (16 per V100); DDP keeps its construction-time gradient buckets, so that a resumed run uses the same reduction order as an uninterrupted one.

B.8 Seeds, pairing and protocol

Random streams are derived from structured SHA-256 keys over experiment, dataset, purpose, replication and component, but not model. Within a replication, the four models therefore start from identical weights for their common modules (encoder, projector and predictor; Gaussian heads for the variational models). They also see identical training examples in identical order, share latent noise (variational models) and share SIGReg source and target directions (Shared models). Replications 1–5 draw from seed streams separate from those of the development replication (ID 0) and of software tests. A check made before main training found accidental shifted-epoch reuse of MPI3D training visits in an earlier seeding scheme, which was repaired. On synthetic test inputs, the repaired scheme gives 500 distinct hashes of the epoch visit orders (five replications $\times$ 100 epochs); distinct hashes

detect accidental reuse but do not prove statistical independence. Torch CPU seeds that differ only above bit 31 give identical draws, so the seed manifest is checked for collisions on the effective low 32 bits. Latent, SIGReg and forecast streams are stateful generators rather than reseeded per step. Evaluation streams are fixed across models and replications.

Before the main runs, one development replication per model and dataset was trained and evaluated. Its review settled the readout transforms and evaluation counts. The evaluation configuration, final query-bank definitions and statistical analysis were then fixed (26 September 2026, 23:38 UTC). All 40 main runs were trained afterwards, and each endpoint is the terminal checkpoint of the declared budget, chosen independently of outcomes.

B.9 Fidelity to the AdaSSL video recipe

Table 10 classifies each component of the AdaSSL reference as supported by the AdaSSL paper, a local convention where the paper leaves a choice open, or a deliberate adaptation.

Table 10: Fidelity of the AdaSSL reference (and of the components the other models share with it) to the AdaSSL video description [Zhang et al., 2026, App. D.5]. The official repository, pinned at commit 8f7148f6, contains no video pipeline. “Local convention” marks a choice that the published text leaves open; each was fixed before any main run.

Status	Components
Supported by the paper	Axis law (equiprobable changed axis, noise proportional to v); $28 \rightarrow 16$ digit on 64 canvas; centre $\mathcal{U}[8, 16]$, velocity $\mathcal{U}[0, 3]$; three-frame segments x, y, u ; 50k/10k/test identity counts; five-layer 3D CNN (32–256 channels, 768-d pooled); projector and predictor widths and BN placement; $d_r = 2$ and head width 1024; posterior sees the source and a later clip; $\beta = 0.001$ constant; EMA 0.996 constant; AdamW, learning rate and weight decay 10^{-4} , cosine schedule; 75k updates at batch 128.
Local convention (unresolved in the reference)	Scale $\frac{2}{3}v$ read as a standard deviation (consistent with AdaSSL’s statement that the new velocity lies within $(-v_0, 3v_0)$ with high probability, where $v_0 = 3$ is the largest initial speed; under this reading that interval is the three-standard-deviation range at $v = v_0$); antialiased bilinear resize and translation with integer pixel centres; cropping without bounce; new velocity from frame 3 $\rightarrow$ 4; spatial strides 2/2/1/1/2 and bias-free convolutions; forward-only negative cosine (the image BYOL code sums two directions); independent target BN statistics with parameter-only EMA; weight decay on all parameters (the image code excludes one-dimensional parameters); scalar reductions of the loss terms.
Deliberate adaptation	Shared models (Shared-Det, Shared-Var) with SIGReg; the MPI3D image/command task; physical readouts and the Energy Score (AdaSSL reports probes and retrieval); paired initialization across models.

C Evaluation and statistics

Energy Score estimators. For Moving-MNIST query i with forecasts $\{\hat{s}_{im}\}_{m=1}^{64}$ and independent truth draws $\{s_{il}\}_{l=1}^{128}$,

$$\widehat{\text{ES}}_i = \frac{1}{64 \cdot 128} \sum_{m,l} \|\hat{s}_{im} - s_{il}\| - \frac{1}{2 \cdot 64 \cdot 63} \sum_{m \neq m'} \|\hat{s}_{im} - \hat{s}_{im'}\|.$$

The self term is zero for a singleton forecast. For MPI3D query i with atoms $\hat{s}_{ik}$ and weights w_k , and outcomes s_{ij} ($j \in \{\text{fail}, \text{succeed}\}$) with weights $\frac{1}{2}$,

$$\text{ES}_i = \sum_{k,j} w_k \frac{1}{2} \|\hat{s}_{ik} - s_{ij}\| - \frac{1}{2} \sum_{k,k'} w_k w_{k'} \|\hat{s}_{ik} - \hat{s}_{ik'}\|.$$

The evaluator computes MPI3D distances as RMS over the two coordinates and multiplies by $\sqrt{2}$ to report Euclidean grid-index distance. The dataset score is the mean over queries.

Gap normalization. In the latent score, forecasts and the two outcome embeddings are unit-normalized, distances are RMS over the 128 coordinates, and each query’s score is divided by $\text{gap}_i = \text{RMS}(e_{i,\text{fail}} - e_{i,\text{succeed}})$. A query is undefined if $\text{gap}_i \leq 10^{-8}$. None was undefined in any run, so “mean” and “mean over defined queries” coincide. Evaluation is deterministic and a failed command leaves the image unchanged, so persistence places all its mass on the failure outcome and scores $\frac{1}{2}$; the true distribution scores $\frac{1}{4}$ (Section 4.3). For any single-point forecast x with outcome embeddings f and s , the triangle inequality gives $\frac{1}{2}(\|x - f\| + \|x - s\|) \geq \frac{1}{2}\|f - s\|$, so its normalized score is at least $\frac{1}{2}$.

Readout. Each checkpoint receives its own ridge readout, fitted on real target-space features: the EMA copy for EMA models, the shared encoder for Shared models. On Moving-MNIST the readout maps real target clips (frames 4–6) to their velocity v' . On MPI3D it maps real images to their position. Features are optionally unit-normalized and then standardized with the fit bank’s mean and standard deviation; coordinates with standard deviation below 10^{-8} are dropped. The weights solve $(Z^\top Z/n + \alpha I)w = Z^\top (Y - \bar{Y})/n$ for each α by eigendecomposition. The transform is raw or unit normalization, and $\alpha \in \{10^{-6}, 10^{-4}, 10^{-2}, 1, 100\}$. The pair (transform, α) with the lowest real-feature selection MSE is kept, with exact ties going to raw. Forecast scores are never used for this selection. The selected transform is applied identically to predicted features. Raw was selected for every run except Moving-MNIST Shared-Det (unit, in all five runs). Encode–decode error is reported as a diagnostic and never subtracted. Predicted-feature support is the standardized norm of each predicted atom under the fitted head, compared with the 99th percentile of the fit features’ norms.

Probes. On Moving-MNIST the digit probe is a linear softmax trained with AdamW (learning rate 10^{-3} , weight decay 0.01, batch 256, at most 100 epochs). It keeps the epoch with the lowest selection cross-entropy and stops after 10 epochs without a decrease of at least 10^{-5} . The velocity probe is an exact affine least-squares fit with rank diagnostics. On MPI3D, affine probes are trained with Adam (batch 512, learning rate 10^{-3} , at most 100 epochs). The penalty grid is $\{0, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$. Penalty and epoch are selected by validation mean position R^2 or balanced accuracy, with ties going to the first penalty and the earliest epoch. Positions are normalized for the position probe. Table 11 lists the information available to each forecast and control.

Table 11: Information available to each forecast and control. On Moving-MNIST, the encode–decode oracle encodes 64 futures drawn from the true law for the query’s digit and source state, not the query’s own truth draws; on MPI3D, it encodes both outcome images.

Forecast / control	Dataset	Information used
Sampled prior (primary)	both	source pixels (and command); trained model; fitted readout
Fixed prior mean	both	as above, residual set to the prior mean
Target-space persistence	both	model’s target-space embedding of the source, decoded
Wrong command / wrong source	MPI3D	rotated command / another query’s source embedding
Pixel constant velocity	Moving-MNIST	source pixels only; centroid displacement, no learning
Encode–decode oracle	both	true future images, encoded and decoded (privileged)
Simulator oracle	Moving-MNIST	64 draws from the true conditional law (privileged)
Coordinate persistence / commanded move	MPI3D	true source coordinates (privileged)
Two-outcome oracle	MPI3D	true outcome distribution (privileged)

Paired interval. For differences $d_1, \dots, d_5$ between Shared-Var and a reference within each replication, the interval is $\bar{d} \pm t_{0.975,4} s_d / \sqrt{5}$ with $t_{0.975,4} = 2.7764$. It assumes that the differences are independent across replications and approximately normal. Each of the four intervals is interpreted on its own.

D Training and inference cost (post hoc, secondary)

Cost was not a prespecified outcome. We counted FLOPs and parameters and ran the timing benchmark after the main runs; the development wall-clock was recorded from the development runs before main training. Table 12 reports absolute values for all four models, Figure 4 shows them relative to the AdaSSL reference, and Table 4 in the main text summarizes the variational-model comparison.

Mechanism. Table 9 counts encoder passes per update from the executed forward methods. The BYOL reference encodes the source with gradients and the target without. The AdaSSL reference adds a differentiable encoding of its posterior input: the later clip u on Moving-MNIST, the target image on MPI3D. Shared-Det encodes source and target with gradients. Shared-Var differs between the datasets. On MPI3D the posterior observes the realized next state, and its input is the target encoding Shared-Var already computes, so it makes the same two passes as Shared-Det. On Moving-MNIST the posterior observes a separate clip, and Shared-Var backpropagates through three passes. The encoder-pass difference follows the posterior inputs.

Table 12: Training cost per model (post hoc, secondary; ↓ lower is better; bold marks the lowest value per column and dataset, without implying a ranking of the models). “Enc. passes” are encoder forward / backward passes per update. Stored parameters and training GFLOPs per update are analytic. Time per update is from a controlled single-GPU benchmark, and wall-clock is from the one development run per model ($n = 1$). Budgets were matched in updates, not in cost. The evidence sources are described below, and Table 13 gives activation memory and the full record.

Model	Enc. passes	Params (M) ↓	GFLOPs ↓	ms / update ↓	Wall-clock (s) ↓
<i>Moving-MNIST</i>					
BYOL reference	2 / 1	6.03	134.2	276	4,662
AdaSSL reference	3 / 2	6.44	234.3	479	8,177
Shared-Det	2 / 2	3.67	200.6	428	7,499
Shared-Var	3 / 3	4.08	300.8	631	11,044
Shared-Var vs. AdaSSL reference		−37%	+28%	+32%	+35%
<i>MPI3D</i>					
BYOL reference	2 / 1	10.70	105.5	57	5,487
AdaSSL reference	3 / 2	11.11	184.4	99	7,411
Shared-Det	2 / 2	6.01	157.8	88	6,088
Shared-Var	2 / 2	6.42	158.0	90	6,511
Shared-Var vs. AdaSSL reference		−42%	−14%	−9%	−12%

Detailed comparison. On MPI3D, Shared-Var needs 14% fewer training FLOPs per update than the AdaSSL reference (158.0 against 184.4 GFLOPs) and 42% fewer stored parameters (6.42M against 11.11M). Its development run took 12% less wall-clock time. The controlled benchmark shows 9% less time per update (90 against 99 ms) at equal saved-activation memory (789 against 787 MiB), since both models keep two differentiable encoder graphs. Adding the residual to Shared-Det costs only 0.1% more FLOPs on MPI3D. On Moving-MNIST, Shared-Var needs 28% more FLOPs per update than the AdaSSL reference (300.8 against 234.3 GFLOPs), 50% more activation memory (1184 against 789 MiB), 32% more benchmark time and 35% more development wall-clock. It stores 37% fewer parameters (4.08M against 6.44M). Within each dataset the FLOP, benchmark and wall-clock measurements agree on the sign of the difference. Forecasting uses only the source encoder, the prior and the predictor, so per-forecast inference cost is identical between an EMA model and the Shared model of the same family. The deployment difference is a checkpoint with one encoder instead of two. Budgets were matched in updates; accuracy at equal training cost and scaling to other architectures were not tested.

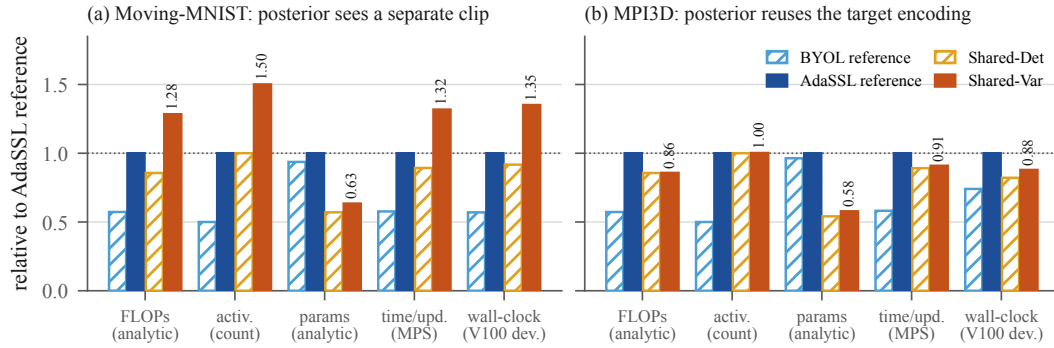


Figure 4: Cost relative to the AdaSSL reference (post hoc, secondary; lower is better; numbers above bars are Shared-Var’s ratios). FLOPs and parameters are analytic counts, activation memory is a saved-tensor count, time per update is from the controlled benchmark, and wall-clock is from one development run per model. The sign of the Shared-Var–AdaSSL reference difference follows the posterior input.

Evidence sources. (1) *Analytic*: parameter counts and training FLOPs per update (forward and backward). Saved-activation memory, the bytes of non-parameter tensors saved for backward, is counted with saved-tensor hooks in the same setting. All three use the final training recipes on

Table 13: Full cost record (post hoc; lower is better in every column; bold marks the lowest value per column and dataset). “Bench.” is the benchmark’s CPU count of training GFLOPs per update, which appears in Table 12. “Math attn.” recounts MPI3D with the math attention backend forced, so that attention matmuls are counted identically in every pass. Under the default backend they are counted in the no-gradient EMA pass but not in gradient-carrying passes; Moving-MNIST has no attention. The recount raises all MPI3D counts by about 6% and leaves the Shared-Var/AdaSSL reference ratio unchanged (0.857 in both). Saved activations are measured on MPS; the CPU recount with the math backend gives the same values to within 1 MiB. Development wall-clock is in Table 12. “Sweep 1 / 2” are the median MPS update times of the two sweeps. Inference MFLOPs are per forecast (number of prior draws or atoms in parentheses), counted with the transformer fast path disabled.

Dataset	Model	Trainable params (M)	Training GFLOPs		Saved act. (MiB)	ms / update sweep 1 / 2	Inference MFLOPs
			bench.	math attn.			
Moving-MNIST	BYOL reference	3.674	134.2	134.2	395	275 / 278	263 (1)
	AdaSSL reference	4.081	234.3	234.3	789	478 / 481	429 (64)
	Shared-Det	3.674	200.6	200.6	790	428 / 428	263 (1)
	Shared-Var	4.081	300.8	300.8	1184	630 / 632	429 (64)
MPI3D	BYOL reference	5.993	105.5	112.0	394	58 / 57	440 (1)
	AdaSSL reference	6.399	184.4	195.6	787	99 / 99	479 (16)
	Shared-Det	5.993	157.8	167.5	788	88 / 88	440 (1)
	Shared-Var	6.399	158.0	167.7	789	90 / 90	479 (16)

synthetic tensors of the real shapes and batch sizes. (2) *Development wall-clock*: the scheduler-reported elapsed time (Slurm ElapsedRaw) of the single development run per model, including start-up, teardown and checkpointing. Moving-MNIST ran on one V100 and MPI3D on four V100s with DDP. (3) *Controlled benchmark*: all eight model–dataset combinations on one Apple-silicon GPU (PyTorch MPS), with 10 warm-up and 100 timed full updates (optimizer, EMA and SIGReg included) per combination, in two sweeps of alternating model order. MPI3D uses its global batch of 64 on the single device, so there is no DDP communication. (4) *Main-run wall-clock*: recorded on the original storage but not retrieved. We make no estimate of it.

Scope and scripts. Analytic counts cover matmul and convolution operations as seen by `torch.utils.flop_counter.FlopCounterMode`. They exclude elementwise, normalization, optimizer, EMA and SIGReg characteristic-function work, and they are not a runtime prediction. Parameter counts agree between the two benchmarks and, on Moving-MNIST, with a capacity count made before training. The MPS benchmark ran PyTorch 2.8.0 on a single Apple-silicon GPU; the main runs used PyTorch 2.5.1 (CUDA 11.8) on V100s. The values come from a benchmarking script, run once on CPU for FLOP counts (one warm-up and two measured updates, inference batch 8) and once on MPS for timing (10 warm-up and 100 timed updates, two sweeps), and from a recount script that forces the math attention backend. Both scripts are part of the codebase. The benchmark’s own inference counts for MPI3D are not used. In evaluation mode the transformer fast path is invisible to the counter, so those counts omit the transformer blocks. The recount disables the fast path.

E Complete results and Moving-MNIST diagnostics

Tables 14–16 and Figures 5–6 give the complete primary results for both datasets. Figures 7–12 and Tables 17–19 give the Moving-MNIST controls, residual diagnostics, probes and readout fidelity. Appendix F gives the MPI3D diagnostics.

Table 14: Primary outcome: physical-forecast Energy Score ($\downarrow$ lower is better), as mean $\pm$ sample SD and range over five training replications. Bold marks the lowest mean per dataset, which by itself does not establish superiority (see Table 15). The datasets have different units. Appendix Table 16 lists all 40 values.

Model	Moving-MNIST ES $\downarrow$ (px/frame)		MPI3D ES $\downarrow$ (grid indices)	
	mean $\pm$ SD	range	mean $\pm$ SD	range
BYOL reference	3.254 ± 0.245	2.866–3.537	$4.04 \times 10^6 \pm 4.03 \times 10^6$	1.00×10^6 – 1.08×10^7
AdaSSL reference	0.777 ± 0.100	0.665–0.881	$4,150 \pm 1,881$	1,773–6,965
Shared-Det	1.625 ± 0.027	1.596–1.659	7.22 ± 2.06	4.54–9.12
Shared-Var	0.702 ± 0.124	0.641–0.923	6.35 ± 2.23	4.25–9.40

Table 15: Prespecified paired contrasts ($\downarrow$ negative values favour Shared-Var). Intervals are individual paired- t intervals ($df = 4$), not simultaneous, and no hypothesis tests were prespecified. “Runs < 0 ” counts the replications in which Shared-Var scored lower.

Dataset	Contrast	Mean diff. $\downarrow$	95% paired t interval	Runs < 0
Moving-MNIST	Shared-Var – AdaSSL reference	−0.075	[−0.320, 0.170]	4 of 5
Moving-MNIST	Shared-Var – Shared-Det	−0.923	[−1.064, −0.782]	5 of 5
MPI3D	Shared-Var – AdaSSL reference	−4,144	[−6,479, −1,809]	5 of 5
MPI3D	Shared-Var – Shared-Det	−0.87	[−3.73, 1.99]	2 of 5

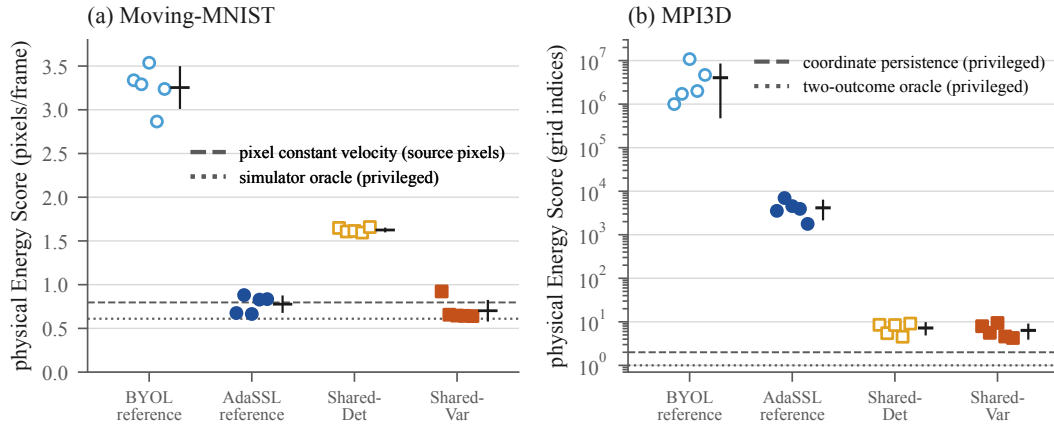


Figure 5: Physical-forecast Energy Score of all 40 runs (lower is better). Within each model, points are replications 1–5 from left to right. Black marks show the mean $\pm$ sample SD; on the MPI3D log axis the lower end is clipped at half the smallest run. Horizontal lines are shared fixed-bank references: pixel constant velocity uses source pixels only, and the other references are privileged.

Table 16: All 40 primary outcomes (physical Energy Score, $\downarrow$) and the per-replication paired differences. Bold marks the lowest score in each replication.

Dataset	Model (ES $\downarrow$)	Rep. 1	Rep. 2	Rep. 3	Rep. 4	Rep. 5
Moving-MNIST	BYOL reference	3.338	3.291	3.537	2.866	3.238
Moving-MNIST	AdaSSL reference	0.676	0.881	0.665	0.829	0.835
Moving-MNIST	Shared-Det	1.649	1.608	1.614	1.596	1.659
Moving-MNIST	Shared-Var	0.923	0.656	0.647	0.643	0.641
Moving-MNIST	Shared-Var – AdaSSL reference	0.247	−0.224	−0.018	−0.186	−0.194
Moving-MNIST	Shared-Var – Shared-Det	−0.726	−0.952	−0.967	−0.953	−1.018
MPI3D	BYOL reference	1.00×10^6	1.72×10^6	1.08×10^7	2.00×10^6	4.68×10^6
MPI3D	AdaSSL reference	3,538	6,965	4,550	3,927	1,773
MPI3D	Shared-Det	8.53	5.48	8.43	4.54	9.12
MPI3D	Shared-Var	7.93	5.58	9.40	4.60	4.25
MPI3D	Shared-Var – AdaSSL reference	−3,530	−6,959	−4,540	−3,922	−1,768
MPI3D	Shared-Var – Shared-Det	−0.60	0.10	0.97	0.06	−4.86

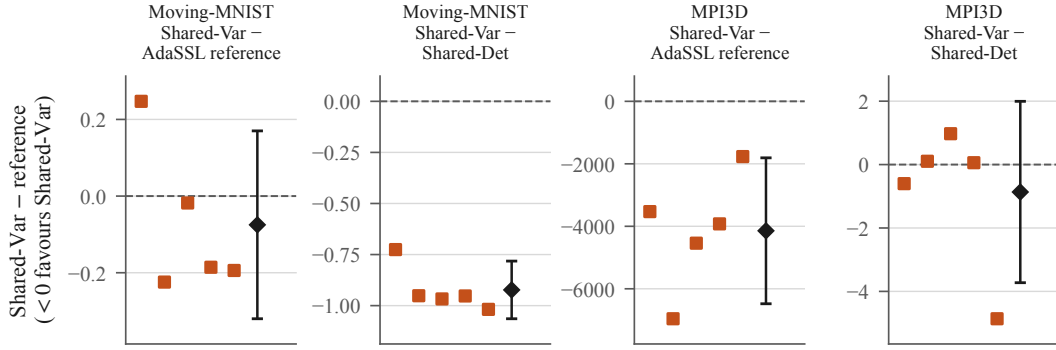


Figure 6: Primary paired contrasts. Squares are replications 1–5 from left to right; the diamond and bar give the mean and the individual paired- t 95% interval. Negative values favour Shared-Var. The scales differ by panel.

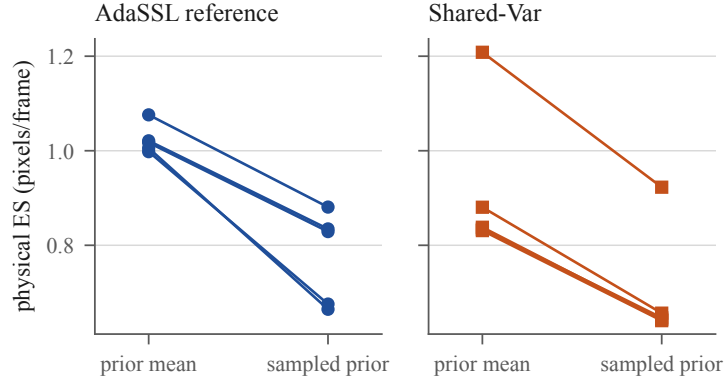


Figure 7: Moving-MNIST sampling control. Each of the five lines per panel joins fixed-prior-mean and sampled forecasts from one trained replication. Sampling lowers physical Energy Score in all ten variational-model runs. This inference control retains the trained encoder, predictor and heads.

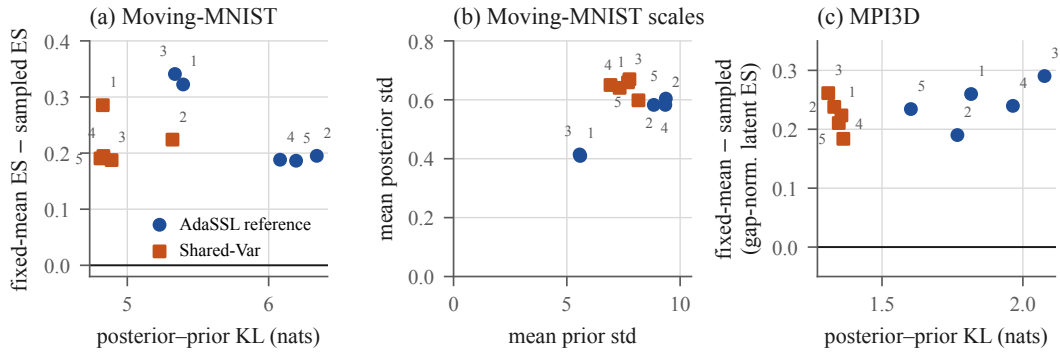


Figure 8: Residual usage at the end of training, per run; numbers are replication IDs. (a) Moving-MNIST: posterior-prior KL on query trajectories against the gain from sampling over the prior mean. (b) Moving-MNIST: mean prior and posterior standard deviations. (c) MPI3D: posterior-prior KL, averaged over the two outcomes and defined for all 2,048 queries, against the sampling gain in the gap-normalized latent score. The posterior is evaluated for diagnosis only and never used to forecast.

Table 17: Moving-MNIST forecasts and controls: physical Energy Score, mean $\pm$ SD over five runs (px/frame). Bold marks the lowest mean in the primary row only; controls computed in separately learned target spaces are not ranked.

Forecast (physical ES $\downarrow$, px/frame)	BYOL reference	AdaSSL reference	Shared-Det	Shared-Var
Sampled prior (primary)	3.254 ± 0.245	0.777 ± 0.100	1.625 ± 0.027	0.702 ± 0.124
Fixed prior mean	–	1.024 ± 0.031	–	0.918 ± 0.163
Target-space persistence	0.836 ± 0.004	0.824 ± 0.004	1.765 ± 0.119	0.863 ± 0.088
Encode–decode oracle	0.632 ± 0.003	0.625 ± 0.002	1.278 ± 0.027	0.652 ± 0.065
Pixel constant velocity		0.797 (one fixed-bank calculation)		
Simulator oracle (privileged)		0.610 (one fixed-bank calculation)		

Table 18: Moving-MNIST representation probes and readout fidelity (means over five runs). Probes use source clips of the final report bank. The forecast readout is fitted on target-space features of real target clips. The support row gives the mean fraction of predicted features beyond the fit features’ 99th-percentile standardized norm. Bold marks the best mean in each row (per axis for x/y cells); it does not establish superiority.

Measurement	BYOL reference	AdaSSL reference	Shared-Det	Shared-Var
Source backbone, digit accuracy $\uparrow$	0.851	0.891	0.878	0.843
Source projector, digit accuracy $\uparrow$	0.706	0.852	0.621	0.517
Source backbone, velocity R^2 (x/y) $\uparrow$	0.974 / 0.969	0.984 / 0.985	0.986 / 0.986	0.991 / 0.990
Source projector, velocity R^2 (x/y) $\uparrow$	0.988 / 0.987	0.992 / 0.995	0.383 / 0.499	0.920 / 0.992
Forecast readout, velocity R^2 (x/y) $\uparrow$	0.973 / 0.972	0.980 / 0.976	0.146 / 0.163	0.904 / 0.976
Encode–decode velocity RMSE (px/frame) $\downarrow$	0.264	0.229	1.540	0.333
Predicted features above fit 99th percentile (fraction) $\downarrow$	0.352	0.000	0.117	0.016
Selected readout transform	raw	raw	unit	raw

Table 19: Moving-MNIST residual diagnostics at the end of training (mean $\pm$ SD over five runs). The KL uses posteriors evaluated on query trajectories for diagnosis only. Intervals are empirical 5–95% marginal intervals from 64 prior draws, evaluated against 128 truth draws. Bold marks the coverage closer to the nominal 0.90.

Endpoint diagnostic	AdaSSL reference	Shared-Var
Posterior–prior KL (nats)	5.87 ± 0.47	4.94 ± 0.22
Mean prior std	7.74 ± 1.99	7.58 ± 0.47
Mean posterior std	0.52 ± 0.10	0.64 ± 0.03
90% interval coverage, x ($\rightarrow 0.90$)	0.679 ± 0.207	0.810 ± 0.169
90% interval coverage, y ($\rightarrow 0.90$)	0.577 ± 0.104	0.879 ± 0.030
90% interval width, x (px/frame)	1.85 ± 0.89	2.19 ± 0.34
90% interval width, y (px/frame)	1.19 ± 0.14	2.35 ± 0.28

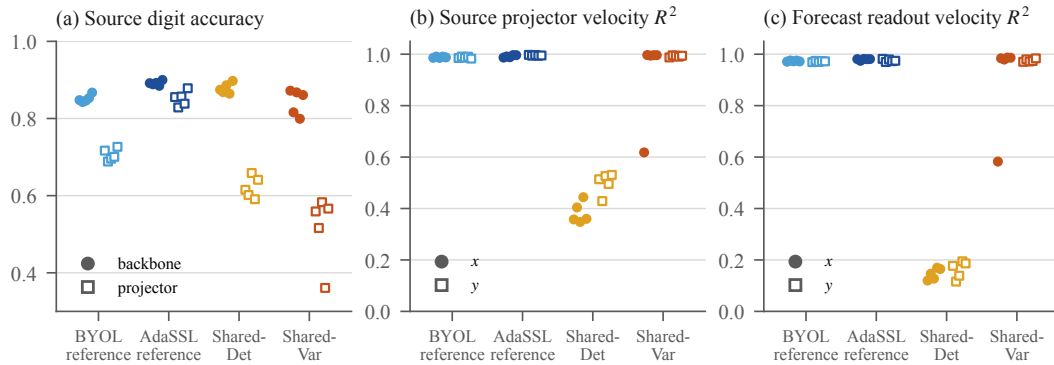


Figure 9: Moving-MNIST representation trade-offs. (a, b) Online source-clip probes. (c) Target-space forecast readout on real target clips. Filled and open markers are the two series named in each legend. Within a series, replications run 1–5 from left to right. Shared-Var’s weak replication 1 is the outlier in (b) and (c); in (a), the low projector digit accuracy (0.361) is replication 4.

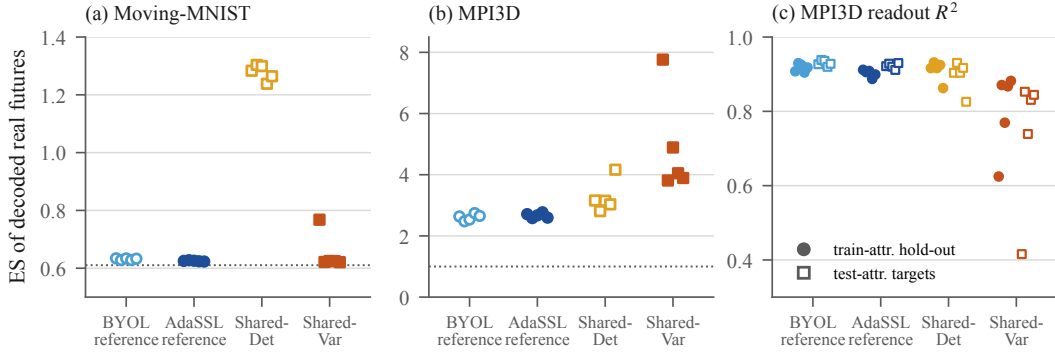


Figure 10: Readout fidelity. (a, b) Energy Score of decoded encodings of real futures (the encode–decode oracle), with the dotted line marking the privileged oracle. (c) MPI3D readout R^2 on the in-support training-attribute hold-out (filled) and on real test-attribute targets (open).

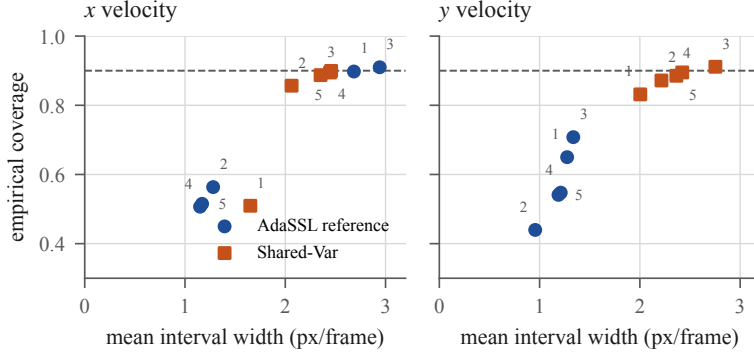


Figure 11: Moving-MNIST empirical 90% marginal interval coverage against mean width, by axis. Points are runs, labelled by replication ID; the dashed line is the nominal level. Deterministic models have zero width and zero coverage by construction. The estimate depends on the number of draws and is not a calibration proof.

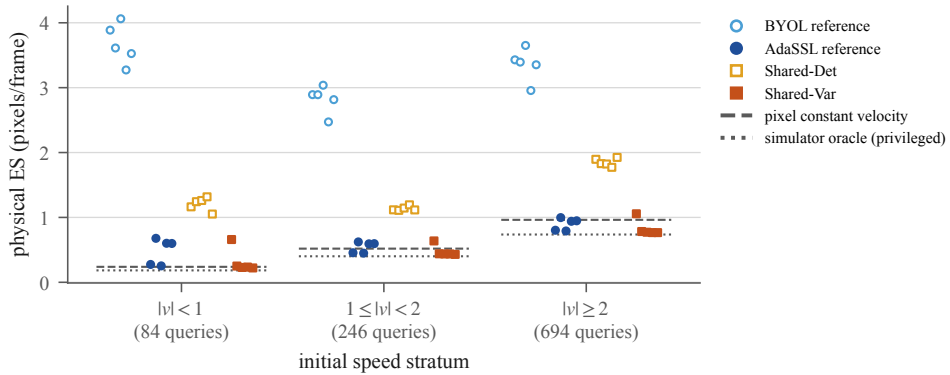


Figure 12: Moving-MNIST physical Energy Score by initial-speed stratum (post hoc, secondary). Points are runs. Lines mark the shared pixel constant-velocity baseline and the privileged simulator oracle, each computed once per stratum on the fixed bank.

F MPI3D diagnostics: no run beats its latent persistence control

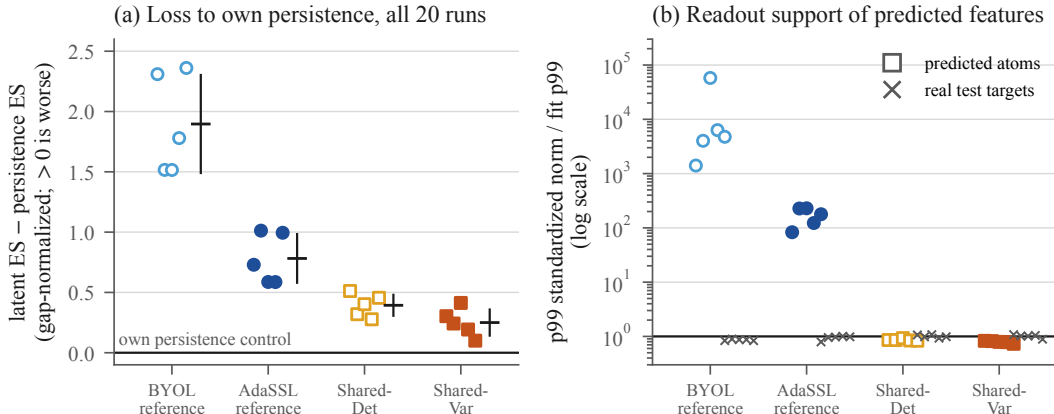


Figure 13: MPI3D failure analysis, all 20 runs. (a) Gap-normalized latent Energy Score minus the same model’s target-space persistence score (exactly 0.5). Values above zero are worse, and every run is above zero; deterministic models cannot fall below zero by construction. Separately learned spaces admit no cross-model ranking. (b) Ratio of the 99th-percentile standardized norm of predicted atoms (model markers) and of real test-attribute targets ($\times$) to that of the readout’s fit features, on a log scale. Every EMA-model predicted atom lies beyond the fit 99th percentile.

Physical scores. In grid indices, Shared-Var scores 6.35 ± 2.23 and Shared-Det 7.22 ± 2.06 . The AdaSSL reference scores $4,150 \pm 1,881$, and the BYOL reference scores $4.04 \times 10^6 \pm 4.03 \times 10^6$. The Shared-Var–Shared-Det difference is -0.87 with interval $[-3.73, 1.99]$, and Shared-Var is lower in only 2 of 5 replications (Figure 6). The interval against the AdaSSL reference excludes zero. That reflects a numerical failure in the EMA models’ pipeline (Figure 13), not a general advantage. Every run of every model is worse than privileged coordinate persistence (2.0).

The EMA models’ predictions leave the readout’s support. In every BYOL reference and AdaSSL reference run, all predicted atoms lie beyond the 99th percentile of the standardized norm of the readout’s fit features. Their mean 99th-percentile norms are 3.03×10^5 and $2,662$, against fit percentiles of 16.2 – 23.3 (Figure 13b). Real test targets stay in support. A check made before main training reproduced these extreme outputs in the development runs with independent ridge solves. A plausible but untested explanation is that the cosine loss leaves the scale of the prediction free, while the readout is fitted on raw real features. The Shared models stay in support. The large physical contrasts describe this pipeline failure; on the latent score, neither Shared model beats its own persistence control.

No model beats persistence, independently of the decoder. In all 20 runs, the gap-normalized latent score is worse than the model’s own persistence control (Figure 13a). The ten deterministic runs cannot score below it by construction (Appendix C); the ten variational runs could, but none does. Shared-Var averages 0.750 (runs: $0.803, 0.742, 0.913, 0.693, 0.600$) and Shared-Det 0.893 , against 0.5 for persistence and 0.25 for the exact distribution. In physical units, Shared-Var beats the decoding of its own persistence (5.92) in 3 of 5 runs, but no run approaches privileged coordinate persistence.

Readout transfer and conditioning controls. On real test-attribute targets, Shared-Var’s readout reaches $R^2 = 0.737$ (replication 1: 0.416). The other models reach 0.90 – 0.93 . Decoding real outcome images scores 4.88 for Shared-Var, against 1.0 for the exact distribution, so the physical scores include substantial readout distortion (Tables 20 and 21). Rotating the command worsens Shared-Var’s latent score in every run (0.920), and a wrong source worsens its physical score to 14.36 . The forecasts therefore depend on source and command, but this sensitivity does not yield prediction beyond persistence. Online representation probes are in Table 22.

Table 20: MPI3D forecasts and controls (means over five runs). The privileged references are single fixed-bank values. Bold marks the lowest mean in the two sampled-prior rows only.

Forecast	BYOL reference	AdaSSL reference	Shared-Det	Shared-Var
<i>Physical Energy Score</i> ↓ (<i>Euclidean grid indices</i>)				
Sampled prior (primary)	4.04×10^6	4,150	7.22	6.35
Fixed prior mean	—	7,397	—	7.47
Target-space persistence	3.80	3.91	4.29	5.92
Wrong command	4.04×10^6	4,150	7.95	6.79
Wrong source	4.04×10^6	4,150	17.64	14.36
Encode-decode oracle	2.61	2.67	3.26	4.88
Coordinate persistence (privileged)		2.000		
Commanded move (privileged)		2.000		
Two-outcome oracle (privileged)		1.000		
<i>Gap-normalized latent Energy Score</i> ↓ (<i>within each model's own target space</i>)				
Sampled prior	2.396	1.282	0.893	0.750
Fixed prior mean	—	1.525	—	0.974
Target-space persistence	0.500	0.500	0.500	0.500
Wrong command	2.403	1.328	1.061	0.920
Wrong source	2.502	1.850	9.045	8.278
Encode-decode oracle	0.250	0.250	0.250	0.250

Table 21: MPI3D readout support and fidelity (means over five runs). Standardized norms use each run's fitted readout and its selected transform. Bold marks the best mean in each directional row; it does not establish superiority.

Diagnostic	BYOL reference	AdaSSL reference	Shared-Det	Shared-Var
Fit features, 99th-percentile standardized norm	23.35	16.18	20.34	19.14
Predicted features, 99th-percentile standardized norm	3.03×10^5	2,662	17.59	15.12
Real test targets, 99th-percentile standardized norm	20.04	15.12	20.50	19.27
Predicted atoms above fit 99th percentile (fraction) ↓	1.0000	1.0000	0.0004	0.0000
Real test targets above fit 99th percentile (fraction) ↓	0.0015	0.0063	0.0155	0.0132
Readout R^2 , train-attribute hold-out ↑	0.917	0.902	0.910	0.803
Readout R^2 , real test-attribute targets ↑	0.930	0.922	0.896	0.737
Selected readout transform	raw	raw	raw	raw

Table 22: MPI3D online representation probes on 2,048 test-attribute images (means over five runs). Bold marks the highest mean in each row; it does not establish superiority.

Features	Probe (↑)	BYOL reference	AdaSSL reference	Shared-Det	Shared-Var
Pooled backbone	Position R^2 (mean of axes)	0.708	0.738	0.899	0.878
	Colour, balanced accuracy	0.977	0.970	1.000	0.999
	Shape, balanced accuracy	0.204	0.220	0.074	0.061
	Size, balanced accuracy	0.824	0.826	0.829	0.818
	Camera height, balanced accuracy	0.999	1.000	1.000	1.000
Projector	Position R^2 (mean of axes)	0.622	0.786	0.897	0.774
	Colour, balanced accuracy	0.921	0.927	0.997	0.997
	Shape, balanced accuracy	0.192	0.231	0.103	0.139
	Size, balanced accuracy	0.798	0.832	0.808	0.798
	Camera height, balanced accuracy	0.997	0.998	1.000	0.998